

CIFAR-100 Image Classification Project

<https://gemini.google.com/app/655dc6a7c3af2a8b>

User prompt: Summarize [https://github.com/MasterSaha/AI-and-ML-Projects/blob/main/Image_classification_on_CIFAR_100_using_any_ML_algorithm_\(in_python\).ipynb](https://github.com/MasterSaha/AI-and-ML-Projects/blob/main/Image_classification_on_CIFAR_100_using_any_ML_algorithm_(in_python).ipynb)

Response: An exact textual breakdown of that specific notebook isn't available due to technical constraints on GitHub's raw code rendering, but we can look at exactly what a project with this name typically achieves.

Based on the title—"Image classification on CIFAR-100 using any ML algorithm"—the project is a machine learning pipeline tailored for the **CIFAR-100 dataset**.

Here is a summary of the typical workflow, architectures, and challenges that this specific notebook addresses:

1. The Dataset (CIFAR-100)

Unlike the simpler CIFAR-10 dataset (which only has 10 classes), CIFAR-100 is notoriously tricky for standard machine learning algorithms.

- **Content:** It contains 60,000 tiny color images (32×32 pixels).
- **Classes:** 100 distinct classes, grouped into 20 "superclasses" (e.g., under *fish*, you have aquarium fish, flatfish, ray, shark, trout).
- **Data Split:** 50,000 training images and 10,000 testing images. Because there are only 500 training images per class, models easily overfit (memorize the data) if not tuned properly.

2. Core Machine Learning Workflow

A Python notebook tackling this problem generally structured into these essential phases:

- **Data Preprocessing:**
 - Loading the dataset (typically via `keras.datasets.cifar100`).
 - **Normalization:** Pixel values are scaled from 0-255 down to 0.0-1.0 or standardized using the dataset's mean and standard deviation. This keeps the optimization math stable.

- **Reshaping:** Standard Machine Learning models (like SVMs or Random Forests) require the 3D images ($32 \times 32 \times 3$) to be flattened into a 1D vector of 3,072 features. Deep learning models keep the 3D shape.
 - **Algorithm Implementations:** The notebook usually explores one of two distinct paths depending on the exact assignment requirement:
 1. **Traditional ML (Scikit-Learn):** Algorithms like *Random Forests*, *Support Vector Machines (SVM)*, or *K-Nearest Neighbors (KNN)*. Because these models struggle with raw pixel patterns, they often achieve lower baseline accuracy (typically around 15–25%) unless paired with advanced feature extraction like HOG (Histogram of Oriented Gradients).
 2. **Deep Learning (TensorFlow/Keras or PyTorch):** A Convolutional Neural Network (CNN). A custom CNN or a transfer learning model (like ResNet or MobileNet) is generally implemented to easily capture spatial patterns, pushing accuracies much higher.
 - **Evaluation:**
 - Uses metrics like a **Confusion Matrix** to see which classes get mixed up.
 - Prints a **Classification Report** featuring Precision, Recall, and F1-Score across the 100 categories.
-

User prompt: Explain the following Code block
`import tensorflow as tf
from tensorflow.keras import datasets, layers, models
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import numpy as np`

Response: This code block handles the **initial setup and imports** needed to build, train, and visualize a Deep Learning model using TensorFlow. It provides the foundational tools required for an image classification task (like the CIFAR-100 project).

Here is a line-by-line breakdown of what each import does:

- `import tensorflow as tf` This is the core library. **TensorFlow** is Google's open-source framework used for numerical computation and building deep learning models. Aliasing it as `tf` is the universal industry standard to keep code clean.
- `from tensorflow.keras import datasets, layers, models` This pulls in the three most critical components of **Keras** (TensorFlow's high-level, user-friendly API):
 - `datasets`: Used to easily download common baseline datasets (like CIFAR-10, CIFAR-100, or MNIST) with a single line of code.
 - `layers`: The building blocks of a neural network (e.g., Convolutional layers (`Conv2D`), Pooling layers (`MaxPooling2D`), and Dense/fully connected layers).

- `models`: Used to define the structure of the network, most commonly the `Sequential` model container, which allows you to stack layers linearly.
- `from tensorflow.keras.utils import to_categorical` This utility is used for **One-Hot Encoding**. Computer vision models struggle to calculate loss directly on text labels or raw integers (like class 3 vs class 4). `to_categorical` converts integer labels into binary vectors.

Example: For 5 classes, an integer label of 2 becomes `[0, 0, 1, 0, 0]`.

- `import matplotlib.pyplot as plt` **Matplotlib** is a plotting library. It is imported here to visualize data—such as printing the actual images from the dataset to look at them, or plotting line graphs of the model's training accuracy and loss over time.
- `import numpy as np` **NumPy** is the fundamental package for scientific computing in Python. Images are treated by computers as multi-dimensional matrices (arrays) of numbers, and NumPy allows you to manipulate, reshape, and process these image matrices incredibly quickly.

User prompt: Explain the following Code block (X_train, y_train), (X_test, y_test) = datasets.cifar100.load_data() X_train = X_train.astype('float32') / 255.0 X_test = X_test.astype('float32') / 255.0 y_train = to_categorical(y_train, 100) y_test = to_categorical(y_test, 100)

Response: This code block handles **loading the dataset** and **preprocessing the data** to get it into the exact format a neural network needs to learn effectively.

Here is exactly what each section is doing:

1. Loading the Data

```
(X_train, y_train), (X_test, y_test) = datasets.cifar100.load_data()
```

This line automatically downloads (if not already downloaded) and splits the CIFAR-100 dataset into two distinct sets:

- **Training Set (X_train, y_train):** 50,000 images and their corresponding labels used to train the model.
- **Testing Set (X_test, y_test):** 10,000 images and labels kept completely separate, used later to evaluate how well the model performs on data it has never seen before.

- X variables hold the **pixel data** (the images), while y variables hold the **numerical labels** (0 through 99 representing the classes).

2. Min-Max Normalization (Scaling Pixel Values)

```
X_train = X_train.astype('float32') / 255.0  
X_test = X_test.astype('float32') / 255.0
```

By default, digital image pixels are stored as integers ranging from 0 (pure black) to 255 (pure white).

- `.astype('float32')` converts these integers into decimal numbers.
- Dividing by 255.0 scales every single pixel down to a value between **0.0 and 1.0**.

Why do this? Neural networks use mathematical optimization algorithms (like Gradient Descent) to learn. If features have large values (like 255), the math can become unstable, leading to exploding gradients. Scaling values to a small range around 0 makes the training process faster and much more stable.

3. One-Hot Encoding the Labels

```
y_train = to_categorical(y_train, 100)  
y_test = to_categorical(y_test, 100)
```

Originally, the labels (y) are simple integers from 0 to 99. However, a neural network's final layer outputs a list of 100 probabilities (one for each class). To calculate how wrong the model is, we need the true labels to match that shape.

`to_categorical` converts those integers into a **binary matrix** (One-Hot encoded vector) of length 100.

- If an image is a "beaver" (let's say class 2), its original label is just 2.
- After this line, it becomes an array of 100 numbers: `[0.0, 0.0, 1.0, 0.0, ... 0.0]`, where only the index of the correct class is marked with a 1.

User prompt: Explain the following Code block
`class_names = ['apple', 'aquarium_fish', 'baby', 'bear', 'beaver', 'bed', 'bee', 'beetle', 'bicycle', 'bottle', 'bowl', 'boy', 'bridge', 'bus', 'butterfly', 'camel', 'can', 'castle', 'caterpillar', 'cattle', 'chair', 'chimpanzee', 'clock', 'cloud', 'cockroach', 'couch',`

```
'crab', 'crocodile', 'cup', 'dinosaur', 'dolphin', 'elephant', 'flatfish', 'forest', 'fox', 'girl', 'hamster', 'house', 'kangaroo', 'keyboard', 'lamp',
'lawn_mower', 'leopard', 'lion', 'lizard', 'lobster', 'man', 'maple_tree', 'motorcycle', 'mountain', 'mouse', 'mushroom', 'oak_tree', 'orange', 'orchid',
'otter', 'palm_tree', 'pear', 'pickup_truck', 'pine_tree', 'plain', 'plate', 'poppy', 'porcupine', 'possum', 'rabbit', 'raccoon', 'ray', 'road', 'rocket', 'rose',
'sea', 'seal', 'shark', 'shrew', 'skunk', 'skyscraper', 'snail', 'snake', 'spider', 'squirrel', 'stock_car', 'sunflower', 'sweet_pepper', 'table', 'tank',
'telephone', 'television', 'tiger', 'tractor', 'train', 'trout', 'tulip', 'turtle', 'wardrobe', 'whale', 'willow_tree', 'wolf', 'woman', 'worm'] plt.figure(figsize=
(10,10))for i in range(16): plt.subplot(4,4,i+1) plt.xticks([]) plt.yticks([]) plt.grid(False) plt.imshow(X_train[i])
plt.xlabel(class_names[np.argmax(y_train[i])]) plt.show()
```

Response: This code block is designed for **data visualization and sanity checking**. Its main goal is to print a clean 4×4 grid of the first 16 images from the training dataset, completely labeled, so you can visually verify that the images loaded correctly and match up with their expected descriptions.

Here is exactly what the code is doing step-by-step:

1. Creating the Human-Readable Class List

The `class_names` list maps the numerical indices (0 through 99) to real English words. In the CIFAR-100 dataset, the raw labels are just integers; index 0 corresponds to 'apple', index 1 to 'aquarium_fish', and index 99 to 'worm'.

2. Setting Up the Grid Canvas

```
plt.figure(figsize=(10,10))
```

This initializes a Matplotlib plotting window and sets its size to 10 by 10 inches, providing plenty of room to display multiple small images clearly without them squeezing into each other.

3. The Visualization Loop

```
for i in range(16):
```

This loop runs exactly 16 times (from `i = 0` to `i = 15`) to draw the first 16 images from your dataset.

- `plt.subplot(4,4,i+1)`: This breaks the 10x10 canvas into a grid of 4 rows and 4 columns. The `i+1` tell Matplotlib which specific slot (1 through 16) to draw the current image into.

- `plt.xticks([])` and `plt.yticks([])`: By passing empty lists, this hides the pixel coordinate numbers along the X and Y axes. This keeps the layout clean, making it look like a photo gallery instead of a mathematical chart.
- `plt.grid(False)`: Turns off any background grid lines over the images.
- `plt.imshow(X_train[i])`: This actually renders the i -th image from your training array onto the current subplot grid space.
- `plt.xlabel(class_names[np.argmax(y_train[i])])`: This places the correct label right beneath the image.

Note on `np.argmax`: Because the previous code block converted your labels into one-hot encoded arrays (like `[0, 0, 1, 0...]`), `np.argmax()` searches that array, finds the index where the 1 is located (in this case, index 2), and passes that index number to your `class_names` list to grab the correct text label (like 'baby').

4. Rendering the Output

```
plt.show()
```

This final command flushes the background buffer and pops open the finished figure directly onto your screen or Jupyter notebook cell output.

User prompt: Explain the following Code block `model = models.Sequential() model.add(layers.Conv2D(32, (3,3), activation='relu', padding='same', input_shape=(32,32,3))) model.add(layers.Conv2D(32, (3,3), activation='relu', padding='same')) model.add(layers.MaxPooling2D((2,2))) model.add(layers.Dropout(0.25)) model.add(layers.Conv2D(64, (3,3), activation='relu', padding='same')) model.add(layers.Conv2D(64, (3,3), activation='relu', padding='same')) model.add(layers.MaxPooling2D((2,2))) model.add(layers.Dropout(0.25)) model.add(layers.Conv2D(128, (3,3), activation='relu', padding='same')) model.add(layers.Conv2D(128, (3,3), activation='relu', padding='same')) model.add(layers.MaxPooling2D((2,2))) model.add(layers.Dropout(0.25)) model.add(layers.Flatten()) model.add(layers.Dense(512, activation='relu')) model.add(layers.Dropout(0.5)) model.add(layers.Dense(100, activation='softmax'))`

Response: This code block builds a **Convolutional Neural Network (CNN)** architecture using Keras. It follows a highly structured, industry-standard pattern for computer vision: **Feature Extraction Blocks** followed by a **Classification Head**.

Let's break down the component layers first, and then trace how the data flows through the model.

The Building Blocks Explained

- `models.Sequential()`: This initializes a blank model container where layers are stacked sequentially, one after another, from top to bottom.
- `layers.Conv2D (Convolutional Layer)`: The core "eye" of the network. It slides small mathematical filters over the image to detect visual features like edges, textures, or shapes.
 - 32, 64, 128: The number of unique filters the layer learns.
 - (3,3): The size of the filter matrix (kernel).
 - `activation='relu'`: Applies the Rectified Linear Unit function, which turns any negative pixel values to 0. This injects non-linearity, allowing the network to learn complex patterns instead of just straight lines.
 - `padding='same'`: Adds a border of zeros around the image before scanning so the spatial dimensions (height and width) don't shrink during convolution.
 - `input_shape=(32,32,3)`: **Only required on the very first layer.** It explicitly tells the network to expect standard CIFAR-100 images (32 × 32 pixels, with 3 color channels: Red, Green, Blue).
- `layers.MaxPooling2D((2,2))`: Downsamples the image by taking the maximum value in every 2 × 2 pixel window. This cuts the image height and width exactly in half, reducing memory usage and forcing the network to focus only on the most dominant features.
- `layers.Dropout(0.25 / 0.5)`: A powerful regularization technique used to prevent **overfitting** (where the model memorizes the training data perfectly but fails on new images). A dropout of 0.25 randomly deactivates 25% of the neurons during each training step. This forces the remaining neurons to work harder and collaborate rather than relying on a few "mvp" pathways.
- `layers.Flatten()`: Flattens the 3D grid of features down into a single, massive 1D vector of numbers, preparing it to transition from image-scanning layers to final decision-making layers.
- `layers.Dense (Fully Connected Layer)`: Standard neural network layers where every neuron connects to every neuron in the previous layer to make high-level decisions.
 - 512: A large hidden layer to weigh all extracted features.
 - 100, `activation='softmax'`: **The output layer.** It outputs exactly 100 values (one for each CIFAR-100 class) wrapped in a **softmax** function, which converts the raw outputs into a clean probability distribution that adds up to 1.0 (100%).

Architecture Overview & Data Flow

Notice how the network is grouped into three distinct blocks that scale up in depth while scaling down in spatial resolution:

Section	Output Shape	What It's Doing
Input Image	(32, 32, 3)	Raw RGB pixel values.
Block 1 (32 filters)	(16, 16, 32)	Looks for low-level features (simple edges, lines, colors). MaxPooling shrinks dimensions from 32×32 to 16×16 .
Block 2 (64 filters)	(8, 8, 64)	Combines edges to see mid-level features (textures, corners, small circles). MaxPooling shrinks dimensions to 8×8 .
Block 3 (128 filters)	(4, 4, 128)	Extracts high-level features (abstract parts like eyes, wheels, leaves). MaxPooling shrinks dimensions to a tiny 4×4 .
Classification Head	(2048) $\rightarrow$ (512) $\rightarrow$ (100)	Flattens the $4 \times 4 \times 128$ grid into 2,048 raw values. The Dense layers reason through these features and output 100 final class probabilities.

This pattern—**doubling the filters while halving the image resolution**—allows the model to process complex global shapes efficiently without blowing up the computer's memory.

User prompt: Explain the following Code block `model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])`
`model.summary()`

Response: This code block handles two separate, crucial tasks: **compiling the model** (giving it a brain and setting the rules for how it will learn) and **printing the architectural blueprint** so you can see exactly how data flows through it.

Part 1: Compiling the Model (`model.compile`)

Before you can feed data into a neural network, you must define its configuration for the learning process. You do this by passing three major components:

- **optimizer='adam'**: The optimizer is the mathematical algorithm that updates the weights of the neural network to minimize errors. **Adam** (short for *Adaptive Moment Estimation*) is the industry-standard go-to optimizer. Think of it like a smart guide driving down a mountain in a fog; it automatically adapts its speed (learning rate), moving quickly when the path is clear and slowing down carefully as it approaches the bottom (the optimal solution).
- **loss='categorical_crossentropy'**: The loss function measures exactly how wrong the model's predictions are. If the model predicts a cat is a truck, the loss value spikes.

Why categorical_crossentropy? Because the labels were converted into one-hot encoded vectors (like `[1, 0, 0...0]`) and there are more than two classes (100 total). It calculates the difference between the model's predicted probability distribution and the true distribution.

- **metrics=['accuracy']**: This defines what you want to track during training so humans can easily understand how well the model is doing. Accuracy simply tracks the percentage of images the model correctly guesses out of the total dataset (e.g., 0.45 means it got 45% correct).

Part 2: Inspecting the Blueprint (`model.summary()`)

This function prints a clean, text-based table directly into the console or notebook cell. It gives you a bird's-eye view of your entire model architecture.

When run, it outputs a table showing:

1. **Layer Names & Types:** The chronological sequence of layers (Conv2D, MaxPooling2D, Dropout, Dense, etc.).
2. **Output Shapes:** The dimensions of the data as it exits each layer. You will visually see the image matrix shrink spatially (e.g., $32 \times 32 \rightarrow 16 \times 16 \rightarrow 8 \times 8$) while growing deeper in channels ($32 \rightarrow 64 \rightarrow 128$).
3. **Param # (Parameters):** The total number of weights and biases that the network needs to learn in each layer.
4. **Total / Trainable Parameters Summary:** At the very bottom, it calculates the sum of all parameters. For the model built in your previous block, this value will sit somewhere around **1.2 to 1.5 million trainable parameters**. This gives you an immediate sense of how "heavy" or memory-intensive your network is before committing time to train it.

User prompt: Explain the following Code block `history = model.fit(X_train, y_train, epochs=100, batch_size=64, validation_split=0.2)`

Response: This code block is where the actual **training process** happens. This is the heavy-lifting phase where the neural network repeatedly looks at the images, makes guesses, calculates how wrong it is, and uses the Adam optimizer to adjust its internal weights to get smarter.

Here is what each parameter inside `model.fit` controls:

- **X_train, y_train**: The training inputs. `X_train` contains the normalized image pixel arrays, and `y_train` holds the corresponding one-hot encoded true labels.
- **epochs=100**: An **epoch** is one complete pass of the network through the entire training dataset. Setting `epochs=100` means the model will look at, learn from, and cycle through the training data 100 distinct times.
- **batch_size=64**: The network doesn't look at all 50,000 images at the exact same time (which would crash the computer's memory), nor does it look at them one by one (which is incredibly slow). Instead, it splits the data into small chunks called **batches**. A batch size of 64 means the model looks at 64 images, calculates the average error, updates its weights once, and then moves on to the next 64 images.
- **validation_split=0.2**: This is a vital setup to check for **overfitting**. Before starting, Keras automatically peels off **20%** of the training data (10,000 images) and sets it aside as a *validation set*, leaving the remaining 80% (40,000 images) for actual weight tuning.

At the end of every single epoch, the model is tested on this hidden 20% validation data. This gives you a real-time health check of how well the model is generalizing to data it isn't actively learning from.

What is history?

The entire command is assigned to a variable named `history`. When training finishes, this object stores a step-by-step diary of the model's performance across all 100 epochs. It saves a dictionary containing four key metrics recorded at the end of every epoch:

1. `loss` (Training error)
2. `accuracy` (Training success rate)
3. `val_loss` (Validation error)
4. `val_accuracy` (Validation success rate)

This `history` object is what developers plug directly into Matplotlib right after this step to draw line graphs tracking the model's learning curves.

User prompt: Explain the following Code block `plt.figure(figsize=(12,5)) plt.subplot(1,2,1) plt.plot(history.history['accuracy'], label='Train Accuracy') plt.plot(history.history['val_accuracy'], label='Validation Accuracy') plt.title('Model Accuracy') plt.xlabel('Epoch') plt.ylabel('Accuracy') plt.legend() plt.subplot(1,2,2) plt.plot(history.history['loss'], label='Train Loss') plt.plot(history.history['val_loss'], label='Validation Loss') plt.title('Model Loss') plt.xlabel('Epoch') plt.ylabel('Loss') plt.legend() plt.show()`

Response: This code block handles **visual diagnostics**. It extracts the historical training log saved in the `history` variable and uses Matplotlib to generate a side-by-side performance chart.

These plots show how well your network learned over its 100 epochs and reveal whether the model is healthy or suffering from **overfitting**.

Step-by-Step Breakdown

- `plt.figure(figsize=(12,5))`: Creates a wide plotting canvas (12 inches wide by 5 inches tall) designed to fit two distinct charts next to each other comfortably.
- `plt.subplot(1,2,1)`: Divides the canvas into a grid with 1 row and 2 columns, and targets the **first slot (left side)**.
 - **The Content**: It plots the training accuracy against the validation accuracy over time.
 - **Goal**: You want to see both lines climbing upward together. If the *Train Accuracy* line reaches 95% but the *Validation Accuracy* line gets stuck at 45% and flattens out, your model is overfitting (memorizing the training images instead of learning broad concepts).
- `plt.subplot(1,2,2)`: Targets the **second slot (right side)** of the grid.
 - **The Content**: It plots the training loss against the validation loss over time.
 - **Goal**: You want to see both lines sloping downward toward zero. If the *Validation Loss* line stops dropping and starts curling upward, it means the model is getting increasingly confident about wrong answers on new data—a classic cue to stop training early.
- **Styling Elements** (`title`, `xlabel`, `ylabel`, `legend`):
 - `title`: Gives each chart a clean heading.

- `xlabel / ylabel`: Labels the axes (X-axis tracks the Epoch from 1 to 100; Y-axis tracks the Accuracy percentage or raw Loss score).
- `plt.legend()`: Spits out a color-coded key in the corner so you know exactly which line represents training and which represents validation.
- `plt.show()`: Renders the complete, finished side-by-side dashboard onto your screen.

What a Healthy vs. Overfitting Chart Looks Like

Because this model uses heavy dropout (up to 50%), the gap between these lines is kept tighter. Here is what you are visually hunting for when looking at this plot:

- **Ideal Behavior**: Both lines smoothly track together. Accuracy trends up; Loss trends down.
- **Overfitting Behavior**: The Training line keeps improving, but the Validation line plateaus or gets worse, leaving a wide, open "alligator mouth" gap between the lines.

User prompt: Explain the following Code block

```
def plot_predictions(index):
    img = X_test[index]
    true_label = class_names[np.argmax(y_test[index])]
    pred_probs = model.predict(np.expand_dims(img, axis=0))
    pred_label = class_names[np.argmax(pred_probs)]
    plt.imshow(img)
    plt.title(f"True: {true_label} | Pred: {pred_label}")
    plt.axis('off')
    plt.show()
for i in range(5):
    plot_predictions(i)
```

Response: This code block handles **visual model evaluation and inference testing**. It wraps the prediction logic into a reusable helper function, loops through the first 5 images of your test dataset, and prints each image side-by-side with its true label versus what the model actually guessed.

Part 1: The Helper Function Breakdown

```
def plot_predictions(index):
```

This defines a function that takes an integer `index` (the position of a specific image in the test set) and processes it through three main steps:

Step A: Extract Data & True Label

```
img = X_test[index]
true_label = class_names[np.argmax(y_test[index])]
```

- It grabs the raw pixel data of the target test image (`img`).
- It looks up the one-hot encoded true label matrix for that image, finds the position of the 1 using `np.argmax()`, and converts that position into the human-readable word (e.g., 'shark') using the `class_names` list we defined earlier.

Step B: Model Inference (The Trickiest Line)

```
pred_probs = model.predict(np.expand_dims(img, axis=0))
pred_label = class_names[np.argmax(pred_probs)]
```

- `np.expand_dims(img, axis=0)`: This is a crucial preprocessing step. When training, the model expects data in batches (e.g., standard input dimensions are `(batch_size, 32, 32, 3)`). A single test image only has the shape `(32, 32, 3)`. Passing it directly causes a shape mismatch crash. This NumPy function adds a dummy batch dimension, reshaping the single image array into `(1, 32, 32, 3)`.
- `model.predict(...)`: The model processes the image and outputs an array of 100 confidence scores (probabilities) that add up to 1.0.
- `np.argmax(pred_probs)`: Finds the index of the highest probability value (the model's top guess) and maps it back to its corresponding string word.

Step C: Plotting

```
plt.imshow(img)
plt.title(f"True: {true_label} | Pred: {pred_label}")
plt.axis('off')
plt.show()
```

- Renders the image on screen.
- Puts a clean title above it comparing reality vs prediction (e.g., True: beaver | Pred: otter).
- Hides the raw coordinate axes values with `plt.axis('off')` so it looks like a clean photograph card.

Part 2: The Loop

```
for i in range(5):  
    plot_predictions(i)
```

This loop executes the function 5 times back-to-back using the indices 0, 1, 2, 3, 4.

When this runs in a Jupyter Notebook, it drops a clean vertical sequence of 5 distinct test images on your screen. This lets you immediately spot whether your model is making smart near-miss mistakes (like mistaking a pickup truck for a tractor) or completely missing the mark.
